Student Performance Prediction & Statistical Analysis System

Project Based Learning (PBL) Report

for the course

Statistics For Machine Learning

BACHELOR OF TECHNOLOGY

IN

CSE-CYBER SECURITY

By

RITHSIKA SULAKHE – 24R11A6236
D. SPANDANA – 24R11A6258
GADDAM NAVYA – 24R11A6262

Under the guidance of
Dr. Hemantha Lakshmi
Associate Professor



Department of CSE-CYBER SECURITY
Accredited by NBA

Geethanjali College of Engineering and Technology
(UGC Autonomous)
(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi)
Cheeryal (V), Keesara (M), Medchal.Dist.-501 301.

A.Y 2026-27 (Sem-I)

Geethanjali College of Engineering & Technology

(UGC Autonomous)

(Affiliated to JNTUH, Approved by AICTE, New Delhi)

Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Accredited by NBA

Certificate

This is to certify that the Project-Based Learning (PBL) report entitled **"Student Performance Prediction & Statistical Analysis System."** has been carried out **Rithsika Sulakhe – 24r11a6236 , D. Spandana – 24r11a6258, Gaddam Navya – 24r11a6262** by of III Year I Semester, B.Tech in CSE-CYBER SECURITY, during the academic year 2026-2027, under the guidance of **Dr. Hemantha Lakshmi**, Department of Computer Science and Engineering. This work is submitted in partial fulfillment of the requirements for the Project-Based Learning (PBL) component of the **Cryptography.**

Internal Guide	**Head of the Department**
Dr. Hemantha Lakshmi	**Dr.Krishna Jyothi**
Associate Professor	**Associate Professor**

Examiner

Signature:

Name:

Designation:

ACKNOWLEDGEMENT

Geethanjali College of Engineering & Technology

(UGC Autonomous)

(Affiliated to JNTUH Approved by AICTE, New Delhi)

Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

DEPARTMENT OF CSE-CYBER SECURITY

Accredited by NBA



DECLARATION BY THE CANDIDATE

We hereby declare that the Project-Based Learning (PBL) report entitled **"Student Performance Prediction & Statistical Analysis System"** submitted to the Department of CSE-CYBERSECURITY, Geethanjali College of Engineering and Technology, is a record of original work carried out by us during the academic year 2026-2027, under the guidance of **Dr. Hemantha Lakshmi,** Department of CSE-CYBER SECURITY. We further declare that this report has not been submitted to any other institution or university for the award of any degree.

Name of the Students:

1. Rithsika Sulakhe – 24R11A6236
2. D. Spandana – 24R11A6258
3. Gaddam Navya – 24R11A6262

Place: _____

Date: _____

TABLE OF CONTENTS

S NO.	Contents	Page No.
1	Abstract	5
2	Introduction	6
3	System Design	8
4	Implementation	20
5	Conclusion	35
6	References	36
7	Program Outcomes	37
8	Program specific Outcome (POS) & Mapping	38
9	Sustainable Develpoment Goals(SDG) &Mapping	39
10	Project Summary	40

1. ABSTRACT

The Student Performance Prediction & Statistical Analysis System is a web-based application designed to help educators monitor student progress and identify learners who may need academic support. With the growing amount of student data collected in colleges, such as attendance, internal assessment marks and study habits, manually analysing this data is time-consuming. This system addresses the problem by providing a platform to record student data, estimate expected performance, and analyse academic trends statistically.

The system uses a transparent weighted formula to calculate a predicted score from three inputs: internal marks (50%), attendance (25%) and daily study hours (2.5 marks per hour). The result is limited to a range of 0 to 100 and is used to classify each student into a Low, Medium or High risk category. The prediction is a rule-based heuristic and not a trained machine-learning model. The system also computes statistical measures, namely mean, median, mode, variance and standard deviation, along with the Pearson correlation coefficient between study hours and final marks. A separate C program implements the same statistical calculations independently, so that the results can be compared with the Java implementation.

The system is implemented using Java 17 for backend processing, with the built-in HttpServer API and no external frameworks. The user interface is built with HTML, CSS and JavaScript, and Chart.js is used for the dashboard charts. A CSV file is used for persistent storage. The application provides a login page, a dashboard with charts, features to add, search, edit and delete student records, server-side input validation, a statistics page and CSV export of the data. Overall, the project demonstrates how simple prediction and statistical analysis can be combined in a full-stack application to support early identification of at-risk students.

2. INTRODUCTION

Student Performance Prediction & Statistical Analysis System

Colleges collect a large amount of data about their students, such as attendance, internal assessment marks, study habits and final examination results. When this data is only stored in registers or spreadsheets, it is difficult to see patterns in it or to notice which students are falling behind before the final exams. The Student Performance Prediction & Statistical Analysis System addresses this problem by providing a single place to record student data, estimate expected performance, and analyse academic trends.

This is fundamentally a data analysis and decision-support problem. The system takes three inputs for each student: attendance, internal marks and daily study hours. It combines them using a weighted formula (50% internal marks, 25% attendance and 2.5 marks per study hour) to calculate a predicted score out of 100. Based on this score, each student is placed in a Low, Medium or High risk category. The system also applies descriptive statistics, namely mean, median, mode, variance and standard deviation, and calculates the Pearson correlation between study hours and final marks. However, the prediction is a rule-based heuristic with fixed weights and not a trained machine-learning model, so it should be used for learning and demonstration, not for real academic decisions.

The project combines these techniques in a web-based application. Users can log in, add, search, edit and delete student records, view a dashboard with charts, check the statistics page and export the data as a CSV file. The system is implemented using Java for the backend (the built-in HttpServer API, with no external frameworks), HTML, CSS and JavaScript for the frontend, Chart.js for the charts, and a CSV file for persistent storage. A separate C program implements the same statistical calculations, so the results can be compared with the Java version.

Objectives

- To provide a web-based system for recording and managing student academic data, with server-side input validation and persistent CSV storage.
- To predict an expected score and risk category from attendance, internal marks and study hours using a transparent weighted formula.
- To calculate statistical measures (mean, median, mode, variance and standard deviation) and the correlation between study hours and final marks, implemented in both Java and C.
- To display results through a dashboard with charts, searchable student records and CSV export, and to document the system’s limitations, including the heuristic nature of the prediction, the small sample size and the demo-level login.

Project Outcomes

1. Student Record Management

- The system stores student records (name, roll number, department, year, attendance, internal marks, study hours and optional final marks) and lets users add, search, edit and delete them.
- The Java backend validates every input (for example, attendance and marks must be 0 to 100) and rejects duplicate roll numbers.
- Records are saved persistently in a CSV file and reloaded when the server restarts.

2. Performance Prediction

- The system calculates a predicted score out of 100 using a weighted formula: 50% internal marks, 25% attendance and 2.5 marks per study hour.
- Each student is classified as Low, Medium or High risk based on the predicted score, which helps identify students who may need attention.

3. Statistical Analysis

- The system calculates the mean, median, mode, variance and standard deviation for predicted scores, recorded final marks and daily study hours.
- It calculates the Pearson correlation coefficient between study hours and final marks and describes the result as a positive, negative or weak association.

4. Independent C Implementation

- A standalone C program implements the same statistical calculations (mean, median, mode, variance, standard deviation and correlation).
- It allows the results to be compared with the Java implementation. On the sample data it reports a mean of 78.17, a standard deviation of 11.29 and a correlation of 0.964.

5. Dashboard and Visualisation

- The dashboard shows the total number of students, the average predicted score, the average attendance and the number of high-risk students.
- A bar chart shows predicted scores by student, and a doughnut chart shows the department mix.
- The data refreshes automatically every 5 seconds, so changes made in another open session appear without a reload.

6. User-Friendly Web Interface

- The website provides a login page, a dashboard, a searchable student table, add/edit forms, a statistics page and CSV export.

3. SYSTEM DESIGN

The System Design of the Student Performance Prediction & Statistical Analysis System describes how student records are entered and validated, how predictions and statistics are calculated, and how results are shown to the user. The system follows a web-based client-server architecture. Users interact with the frontend in the browser, while the Java backend processes requests, applies the prediction formula and statistical calculations, and maintains the stored student records.

3.1. System Architecture Overview

The system operates in three stages:

- **Record Management:** The user logs in and enters a student's details (name, roll number, department, year, attendance, internal marks, study hours and optional final marks). The backend validates the input, rejects duplicate roll numbers and saves the record to the CSV file.

- **Prediction and Risk Classification:** For each student, the backend calculates a predicted score using the weighted formula (0.50 × internal marks + 0.25 × attendance + 2.5 × study hours), limited to 0 to 100. The score is then classified as Low risk (75 or above), Medium risk (50 or above) or High risk (below 50).

- **Statistical Analysis and Display:** The backend calculates the mean, median, mode, variance, standard deviation and the study hours vs final marks correlation. The results are displayed on a dashboard with charts and on a statistics page. The frontend refreshes the data every 5 seconds.

3.2. Components of the System

1. User Interface:

- Provides a web interface developed using HTML, CSS and JavaScript, with Chart.js for the charts.

- Includes a login page, a dashboard, a searchable student table, add/edit forms and a statistics page.

- Displays predicted scores and risk levels, and supports CSV export of the student data.

2. Backend Server:

- Implemented in Java 17 using the built-in HttpServer API, with no external frameworks.

- Handles requests from the frontend through a REST API (login, logout, students, dashboard, stats and export).

- Serves the static frontend files and handles multiple requests using a thread pool.

3. Prediction Module:

- Calculates the predicted score from internal marks, attendance and study hours using a fixed weighted formula.

- Assigns the Low, Medium or High risk category from the predicted score.
- Works as a transparent heuristic and not as a trained machine-learning model.

4. Statistics Module:

- Calculates the mean, median, mode, population variance and standard deviation for predicted scores, final marks and study hours.
- Calculates the Pearson correlation coefficient between study hours and final marks.
- A separate C program implements the same calculations independently, so its results can be compared with the Java implementation.

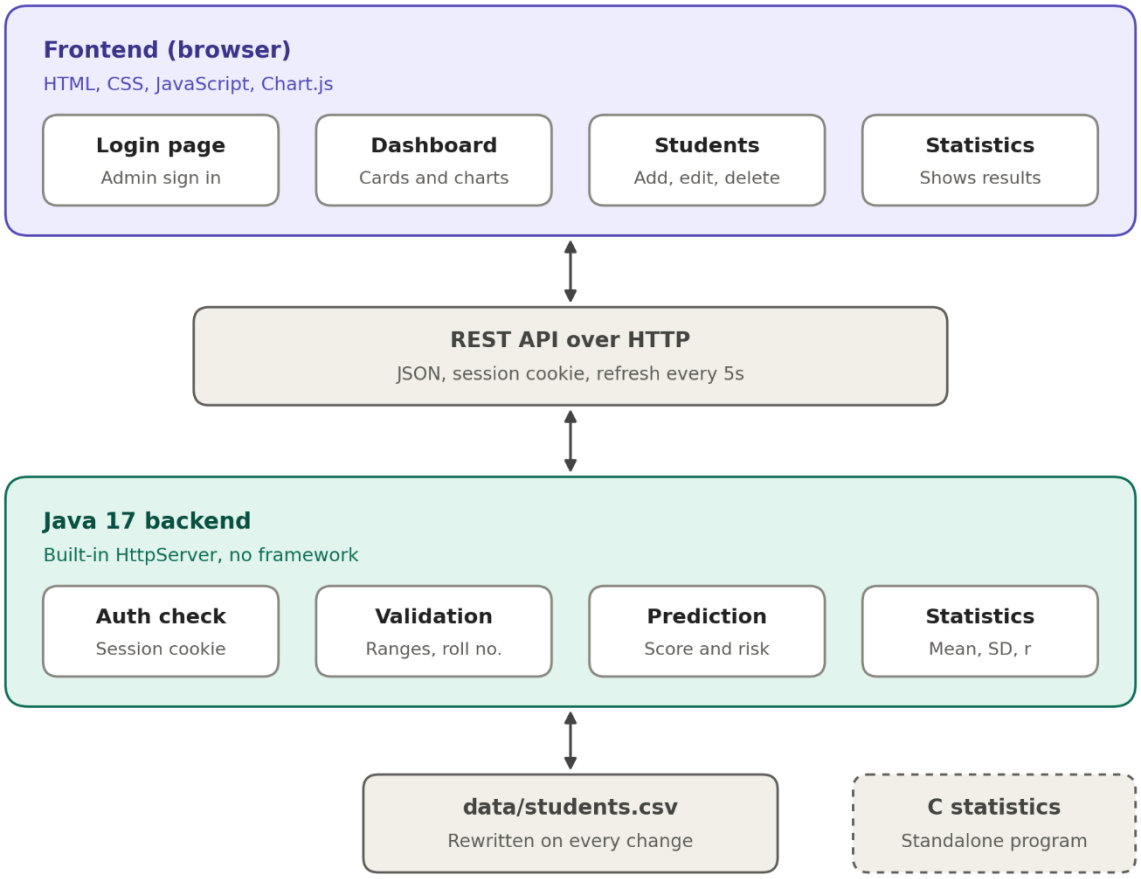
5. Authentication and Validation Module:

- Verifies the login credentials and creates a session token that is sent to the browser as an HttpOnly cookie.
- Checks the session on every API request except login, and returns an error if the user is not logged in.
- Validates all input on the server (for example, attendance and marks must be between 0 and 100) and rejects duplicate roll numbers.

6. Data Storage:

- Uses a comma-separated values (CSV) file (data/students.csv) for persistent storage of student records.
- Loads the records when the server starts, and rewrites the file after every add, edit or delete.
- Creates the file with sample records on the first run if it does not exist.

System Architecture



3.2. Modules

1. Prediction Module (Student class in Main.java)

- This module defines the student record and calculates the predicted score and risk level for each student.
- It holds the data of one student: id, name, roll number, department, year, attendance, internal marks, study hours, optional final marks and the creation time.

Functions:

- Calculate the predicted score as 0.50 × internal marks + 0.25 × attendance + 2.5 × study hours, limited to the range 0 to 100.
- Assign the risk level: Low (75 or above), Medium (50 or above) or High (below 50).
- Convert a student record to JSON for the API, with the predicted score rounded to 2 decimals, and to a CSV row for storage.

2. Storage Module (CSV persistence in Main.java)

- This module keeps the student records in memory and saves them to data/students.csv.
- The file is created automatically on the first run, so the system works without any setup.

Functions:

- Load all records from the CSV file when the server starts, and skip malformed rows.
- Create the data folder and seed 6 sample students if the file does not exist.
- Rewrite the whole file after every add, edit or delete.
- Quote CSV fields correctly (escaping quotes and commas) and parse them back when loading.
- Generate the CSV text used by the Export CSV button.

3. Server Module (Main.java)

- This is the network entry point of the system.
- It runs the built-in Java HTTP server (com.sun.net.httpserver), receives requests from the browser, applies the logic of the other modules and returns JSON.

Functions:

- Serve the web page and the REST API: /api/login, /api/logout, /api/students (GET and POST), /api/students/{id} (PUT and DELETE), /api/dashboard, /api/stats and /api/export.
- Check the login (admin user), create a session token and send it as an HttpOnly cookie. Every API route except login requires a valid session.
- Return clear error replies: 400 for invalid input, 401 when not logged in, 404 when a student or route is not found, 409 for a duplicate roll number and 500 for server errors.
- Serve the static frontend files with protection against path traversal, and handle several requests at once with a thread pool.
- Start on port 8080, or on a port given as a command-line argument.

4. Validation and JSON Helper Module (Main.java)

- This module checks every record before it is saved and handles the JSON text, so the project needs no external library.

Functions:

- Check required fields and number formats, and report clear messages such as "Attendance must be 0–100".
- Enforce the limits: name 2 to 80 characters, roll number up to 30 characters, department 2 to 60 characters, year 1 to 6, attendance and internal marks 0 to 100, study hours 0 to 24 and final marks 0 to 100 (optional).
- Read the JSON body of a request into key-value pairs.
- Escape quotes, backslashes and line breaks so that names and messages are safe inside JSON replies.

5. Statistics Module (Java, Main.java)

- This module calculates the descriptive statistics and the dashboard figures.

Functions:

- Calculate the count, mean, median, mode, population variance and standard deviation for predicted scores, recorded final marks and daily study hours.
- Return no mode when no value repeats or when there is a tie.
- Calculate the Pearson correlation between study hours and final marks, using only students who have final marks.
- Build the dashboard summary: total students, average predicted score, average attendance, number of high-risk students, average final marks and the student count per department.

6. C Statistics Module (c-stats/statistics.c)

- This is a standalone C program that implements the same statistical calculations independently of the Java server.
- It is used to compare the C results with the Java results.

Functions:

- Implement the mean, median (using qsort), mode, variance, standard deviation and correlation as separate functions.
- Run on sample arrays defined in main() and print the results to the console, for example mean 78.17, standard deviation 11.29 and correlation 0.964.
- Compile with gcc statistics.c -o statistics -lm.

7. Web Interface Module (frontend/index.html, app.js, style.css)

- This module presents the system to the user.
- It is a single page application written in HTML5, CSS3 and vanilla JavaScript, with Chart.js for the charts.

Functions:

- Provide a login page and three pages: Overview, Students and Statistics.

- Show the dashboard cards, a bar chart of predicted scores (coloured by risk level) and a doughnut chart of the department mix.
- Provide a searchable student table, an add/edit form, a delete confirmation and a CSV export button.
- Show the mean, median, mode, variance, standard deviation and correlation on the Statistics page.
- Refresh the data every 5 seconds, and escape all displayed text to prevent script injection.

8. Launcher Scripts (run.sh and run.bat)

- These scripts compile the source file and start the server, on Linux/macOS and on Windows.

Functions:

- Create the output folder, run javac, then start the server.
- On Windows, show an error message if the Java compilation fails.

3.3. Message Format

Request and reply format

S No	Field	Description
1	Request line	POST /api/login and POST /api/logout handle the session. GET /api/students, GET /api/dashboard, GET /api/stats and GET /api/export carry no body. POST /api/students adds a student, PUT /api/students/{id} edits one and DELETE /api/students/{id} removes one.
2	Content-Type header	application/json for requests that carry data. The export reply uses text/csv instead.
3	Cookie header	SPSID=<token>, the session cookie set at login (HttpOnly, SameSite=Strict). Every API request except login needs a valid token, otherwise the reply is 401.
4	Body	A JSON object. Login sends username and password. Add and edit send name, rollNo, department, year, attendance, internalMarks, studyHours and an optional finalMarks (blank or null if not recorded).
5	Reply	A JSON body. A student reply contains all the stored fields plus predictedScore and risk. /api/dashboard returns the summary figures, and /api/stats returns the statistics. The reply code is 201 for a new student and 200 otherwise.
6	Error reply	{"error": "..."} with code 400 (invalid input), 401 (not logged in or wrong password), 404 (student or route not found), 409 (duplicate roll number) or 500 (server error).

Student record format (data/students.csv)

S No	Field	Description
1	Id	Unique identifier of the record (a UUID generated by the server).
2	Name	Student's full name, 2 to 80 characters.
3	Roll no	Roll number, up to 30 characters. It must be unique (the comparison ignores case).
4	Department	Department name, 2 to 60 characters (for example CSE, ECE, IT)
5	Year	Academic year, a whole number from 1 to 6.
6	Attendance	Attendance percentage, a whole number from 0 to 100.
7	Internal Marks	Internal marks out of 100, a whole number from 0 to 100
8	Study Hours	Study hours per day, a whole number from 0 to 24.
9	Final Marks	Final marks from 0 to 100 (decimal allowed). Empty if not yet recorded.
10	Created At	UTC timestamp of when the record was created.

3.4. Results

Risk levels (from the predicted score)

S No	Result	Meaning
1	Low risk	Predicted score is 75 or above. The student is expected to perform well.
2	Medium risk	Predicted score is 50 or above but below 75. The student may need monitoring.
3	High risk	Predicted score is below 50. The student needs attention. These students are counted in the "Needs attention" card on the dashboard.

Statistical results

SNo.	Result	Meaning
1	Positive association	The correlation between study hours and final marks is above 0.3. Students who study more tend to score higher in this sample.
2	Negative association	The correlation is below −0.3.
3	Weak or no linear association	The correlation is between −0.3 and 0.3.
4	No unique mode	No value repeats, or two or more values tie for the highest count. The page shows "—" and the C program prints "no unique mode".

Response codes

S N0.	Code	Meaning
1	200	The request succeeded (login, read, edit, delete, statistics, export).
2	201	A new student record was created.
3	400	An input value is missing or invalid (for example, “Attendance must be 0–100”)
4	401	The user is not logged in, or the username or password is wrong.
5	404	The student or API route was not found.
6	409	A student with the same roll number already exists.
7	500	Unexpected server error.

3.5. Threat Model

What the system protects against:

- Invalid or out-of-range data, because the server validates every field before saving a record (for example, attendance and marks must be 0 to 100, and study hours 0 to 24). Invalid input is rejected with a 400 error.
- Duplicate students, because a roll number that already exists (case-insensitive) is rejected with a 409 error.
- Access without logging in, because every API route except login needs a valid session token. Without one, the server replies with 401.
- Script injection through displayed text, because the frontend escapes all student names, roll numbers and departments before showing them (the esc() function in app.js).
- Session cookie theft and cross-site requests, because the cookie is marked HttpOnly and SameSite=Strict, so page scripts cannot read it and other sites cannot send it.

- Access to files outside the frontend folder, because the static file handler rejects any path that leaves it (protection against path traversal).
- Accidental deletion, because the interface asks for confirmation before a record is deleted.
- Damaged CSV rows, because malformed rows are skipped when the file is loaded, so the server still starts.

What it does not stop:

- Anyone who learns the login. The username and password are hardcoded (admin / admin123) and prefilled on the login page. Passwords are not hashed, there is no limit on wrong attempts, and sessions never expire.
- Eavesdropping, because traffic is plain HTTP and not encrypted. HTTPS would be needed in production.
- Direct editing of data/students.csv. The file has no checksum or signature, so changes made outside the application are not detected.
- Unauthorised actions by a logged-in user. There is only one role, so any logged-in user can add, edit or delete any record.
- Loss of history. Edits overwrite the old values, deleted records are gone permanently, and there is no audit log or backup. Editing a record also resets its createdAt time.
- Conflicts between simultaneous requests, because the in-memory student list is not synchronised between server threads.
- Oversized or repeated requests, because the server sets no limit on request size and has no rate limiting.
- Spreadsheet formula injection, because the CSV export does not neutralise values that start with =.
- Wrong decisions from the prediction. It is a fixed-weight heuristic, not a trained model. On the sample data it predicts about 14 points below the actual final marks, so it should not be used for real academic decisions.
- Privacy risks with real data. The README advises against using real student data without proper authentication, encryption and privacy controls.

3.6. Concepts Used

- Weighted scoring (heuristic prediction): the predicted score is a weighted sum of three inputs, 0.50 × internal marks + 0.25 × attendance + 2.5 × study hours. The weights are fixed and chosen by hand, so the prediction is transparent but not learned from data.

- Clamping and threshold classification: the score is limited to the range 0 to 100, then compared with thresholds (75 and 50) to give the Low, Medium or High risk level.

- Mean, median and mode: the average, the middle value after sorting, and the most frequent value. The mode is reported only when exactly one value repeats the most.

- Variance and standard deviation: variance is the average squared distance from the mean, and standard deviation is its square root. The project uses the population formula (dividing by n), which the C program notes in a comment.

- Pearson correlation coefficient (r): measures the linear relationship between study hours and final marks, from −1 to +1. The interface calls it positive above 0.3, negative below −0.3, and weak otherwise. Correlation does not prove causation, and a sample of 6 students is too small for strong conclusions.

- Server-side validation: the server checks every input, whatever the browser sends, so invalid data cannot be stored.

- Session-based authentication: a random UUID token is created at login, kept in server memory and sent to the browser as an HttpOnly cookie.

- REST API and polling: the frontend and backend communicate with JSON over HTTP, and the browser requests fresh data every 5 seconds.

- Persistent storage in CSV: all records are held in memory and rewritten to data/students.csv after every change.

- Future scope, not implemented: fitting the weights with linear regression on the final marks (a real supervised machine learning model), using sample variance, hashing passwords and adding session expiry, HTTPS, a database with backups, role-based access, thread-safe storage, and making the C program read students.csv.

4. IMPLEMENTATION

The Student Performance Prediction & Statistical Analysis System is implemented in Java using only the standard JDK, with no framework or external Java library. The built-in com.sun.net.httpserver package provides the HTTP server, and student records are stored in a plain CSV file. The statistics are also implemented separately in C using only the standard library. The user interface is written in HTML5, CSS3 and vanilla JavaScript, with Chart.js loaded from a CDN for the charts. The main source files of the prediction, storage, server, statistics and interface modules are listed below.

Main.java — Student record, prediction and risk level

```
static class Student {

   String id, name, rollNo, department, createdAt;

   int year, attendance, internalMarks, studyHours;

   Double finalMarks;

   constructor, json() and csv() methods omitted

   double prediction() { return clamp(0.50*internalMarks + 0.25*attendance + 2.5*studyHours, 0, 100); }

   String risk() { double p=prediction(); return p>=75?"Low":p>=50?"Medium":"High"; }

}


static double clamp(double v,double lo,double hi){return Math.max(lo,Math.min(hi,v));}

static double round(double d){return Math.round(d*100.0)/100.0;}
```

Main.java — Server-side validation

```
static void validate(Student s) {

   if(s.name.length()<2 || s.name.length()>80) throw new IllegalArgumentException("Name must be 2–80 characters");

   if(s.rollNo.length()>30) throw new IllegalArgumentException("Roll number is too long");
```

```
    if(s.department.length()<2 || s.department.length()>60) throw new IllegalArgumentException("Enter a valid department");

    if(s.year<1||s.year>6) throw new IllegalArgumentException("Year must be between 1 and 6");

    if(s.attendance<0||s.attendance>100) throw new IllegalArgumentException("Attendance must be 0–100");

    if(s.internalMarks<0||s.internalMarks>100) throw new IllegalArgumentException("Internal marks must be 0–100");

    if(s.studyHours<0||s.studyHours>24) throw new IllegalArgumentException("Study hours must be 0–24");

    if(s.finalMarks!=null&&(s.finalMarks<0||s.finalMarks>100)) throw new IllegalArgumentException("Final marks must be 0–100");

}
```

Main.java — CSV storage: load, save and parse

```
static void loadData() throws IOException {

    if(!Files.exists(CSV)){seed();saveData();return;}

    List<String> lines=Files.readAllLines(CSV,StandardCharsets.UTF_8);

    for(int i=1;i<lines.size();i++){

        List<String> c=parseCsv(lines.get(i));

        if(c.size()<10)continue;

        try{

            Student s=new Student(c.get(0),c.get(1),c.get(2),c.get(3),Integer.parseInt(c.get(4)),

                Integer.parseInt(c.get(5)),Integer.parseInt(c.get(6)),Integer.parseInt(c.get(7)),

                c.get(8).isBlank()?null:Double.parseDouble(c.get(8)),c.get(9));

            students.put(s.id,s);
```

```
        }catch(Exception ignored){}
    }
}
static void saveData() throws IOException {Files.writeString(CSV,csvAll(),StandardCharsets.UTF_8);}
static String csvq(String s){return "\""+s.replace("\"","\"\"")+"\"";}
static List<String> parseCsv(String line){
    List<String>out=new ArrayList<>();StringBuilder b=new StringBuilder();boolean in=false;
    for(int i=0;i<line.length();i++){
        char c=line.charAt(i);

if(c=='"'){if(in&&i+1<line.length()&&line.charAt(i+1)=='"'){b.append('"');i++;}else in=!in;}
        else if(c==','&&!in){out.add(b.toString());b.setLength(0);}
        else b.append(c);
    }
    out.add(b.toString());return out;
}
```

Main.java — Server start, login and session check

```
public static void main(String[] args) throws Exception {
    if (args.length > 0) PORT = Integer.parseInt(args[0]);
    Files.createDirectories(DATA_DIR);
    loadData();
    HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
```

```
    server.createContext("/", Main::handle);
    server.setExecutor(Executors.newCachedThreadPool());
    server.start();
    System.out.println("Student Performance System running at http://localhost:" + PORT);
}

 inside handle(): login, then every other /api/ route requires a session
if (path.equals("/api/login") && method.equals("POST")) {
    Map<String,String> b=body(ex);
    if ("admin".equals(b.get("username")) && "admin123".equals(b.get("password"))) {
        String token=UUID.randomUUID().toString(); sessions.add(token);
        ex.getResponseHeaders().add("Set-Cookie","SPSID="+token+"; Path=/; HttpOnly; SameSite=Strict");
        send(ex,200,"{\"ok\":true,\"message\":\"Welcome back\"}"); return;
    }
    send(ex,401,"{\"error\":\"Invalid username or password\"}"); return;
}
if (path.startsWith("/api/")) {
    if (!authenticated(ex)) { send(ex,401,"{\"error\":\"Please log in\"}"); return; }
    // /api/students, /api/dashboard, /api/stats and /api/export handled here
}
 error replies
catch(IllegalArgumentException e) { send(ex,400,"{\"error\":"+q(e.getMessage())+"}");
}
```

```
catch(Exception e) { e.printStackTrace(); send(ex,500,"{\"error\":\"Server error\"}"); }
```

Main.java — Statistics: mean, variance, median, mode and correlation

```
List<Double> s=new ArrayList<>(a); Collections.sort(s);
double mean=s.stream().mapToDouble(x->x).average().orElse(0), var=0;
for(double x:s) var+=(x-mean)*(x-mean);
var/=s.size();                          // population variance
Map<Double,Integer> freq=new HashMap<>();
for(double x:s) freq.put(x,freq.getOrDefault(x,0)+1);
int max=0; for(int count:freq.values()) max=Math.max(max,count);
Double mode=null; int modes=0;
if(max>1){
   for(Map.Entry<Double,Integer>                                    e:freq.entrySet())
if(e.getValue()==max){mode=e.getKey();modes++;}
   if(modes!=1) mode=null;                  // tie -> no unique mode
}
double med=s.size()%2==1 ? s.get(s.size()/2) : (s.get(s.size()/2-1)+s.get(s.size()/2))/2;
```

standard deviation = Math.sqrt(var); the values are then returned as JSON

```
static double correlation(Collection<Student> ss){
   List<Student> a=new ArrayList<>();
   for(Student s:ss) if(s.finalMarks!=null) a.add(s);
   if(a.size()<2) return 0;
   double x=0,y=0;
```

```
    for(Student s:a){x+=s.studyHours;y+=s.finalMarks;}
    x/=a.size(); y/=a.size();
    double num=0,dx=0,dy=0;
    for(Student s:a){
        num+=(s.studyHours-x)*(s.finalMarks-y);
        dx+=Math.pow(s.studyHours-x,2);
        dy+=Math.pow(s.finalMarks-y,2);
    }
    return dx==0||dy==0 ? 0 : num/Math.sqrt(dx*dy);
}
```

statistics.c — standalone C statistics module (compile: gcc statistics.c -o statistics -lm)

```
static int compare_double(const void *a, const void *b) {
    double x = *(const double *)a, y = *(const double *)b;
    return (x > y) - (x < y);
}
static double mean(const double a[], int n) {
    double sum = 0; for (int i=0;i<n;i++) sum += a[i]; return n ? sum/n : 0;
}
static double median(double a[], int n) {
    if (!n) return 0;
    qsort(a,n,sizeof(double),compare_double);
    return n%2 ? a[n/2] : (a[n/2-1]+a[n/2])/2.0;
}
```

```
static double variance(const double a[], int n) {
   if(!n) return 0; double m=mean(a,n), sum=0;
   for(int i=0;i<n;i++) sum+=(a[i]-m)*(a[i]-m);
   return sum/n; /* population variance */
}
static double correlation(const double x[], const double y[], int n) {
   if(n<2) return 0; double mx=mean(x,n),my=mean(y,n),num=0,dx=0,dy=0;
   for(int  i=0;i<n;i++){num+=(x[i]-mx)*(y[i]-my);dx+=(x[i]-mx)*(x[i]-mx);dy+=(y[i]-my)*(y[i]-my);}
   if(dx==0||dy==0)return 0;return num/sqrt(dx*dy);
}
```

mode() and main() omitted; main() runs these on sample arrays, for example:

double scores[] = {88,76,61,94,68,82}; double hours[] = {3,2,1,4,2,3};

Javascript

app.js — API helper, auto refresh and output escaping

```
async function api(path,options={}){
   const res=await fetch(path,{credentials:'same-origin',headers:{'Content-Type':'application/json',...(options.headers||{})},...options});
   let data;
   if((res.headers.get('content-type')||'').includes('json')) data=await res.json(); else data=await res.text();
   if(!res.ok) throw new Error(data.error||'Request failed');
   return data;
}
```

```
async function refreshAll(silent=false)
{
   try{
      const latest=await api('/api/students');
      const signature=JSON.stringify(latest);
      if(signature!==lastDataSignature){          // re-render only when the data changed
         students=latest; lastDataSignature=signature;
         renderStudents(); await loadDashboard();
         if(!$('statisticsPage').classList.contains('hidden')) await loadStats();
      }
   }catch(e){if(!silent)toast(e.message)}
}
function                                                                    esc(v){return
String(v??").replace(/[&<>"']/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#3
9;'}[c]))
}
```

/Poll the Java backend every 5 seconds so other open sessions see changes too.

setInterval(()=>{if(!$('appView').classList.contains('hidden'))refreshAll(true)},5000);

4.1 SAMPLE OUTPUT

The following output is from a sample run with the 6 seeded students. The server was started, the admin user logged in, the dashboard and statistics were requested, and two invalid requests were rejected. Finally, the C statistics program was run.

[server] Student Performance System running at http://localhost:8080

[server] Demo login: admin / admin123

POST /api/login -> 200 {"ok":true,"message":"Welcome back"}

GET /api/students -> 401 {"error":"Please log in"} (request without a session)

GET/api/dashboard->200
{"totalStudents":6,"averagePrediction":63.83,"averageAttendance":80.67,
"atRisk":1,"averageFinalMarks":78.17,"departments":{"CSE":3,"ECE":2,"IT":1}}

POST /api/students -> 400 {"error":"Attendance must be 0–100"} (attendance = 120)

POST /api/students -> 409 {"error":"Roll number already exists"} (roll number "cse001")

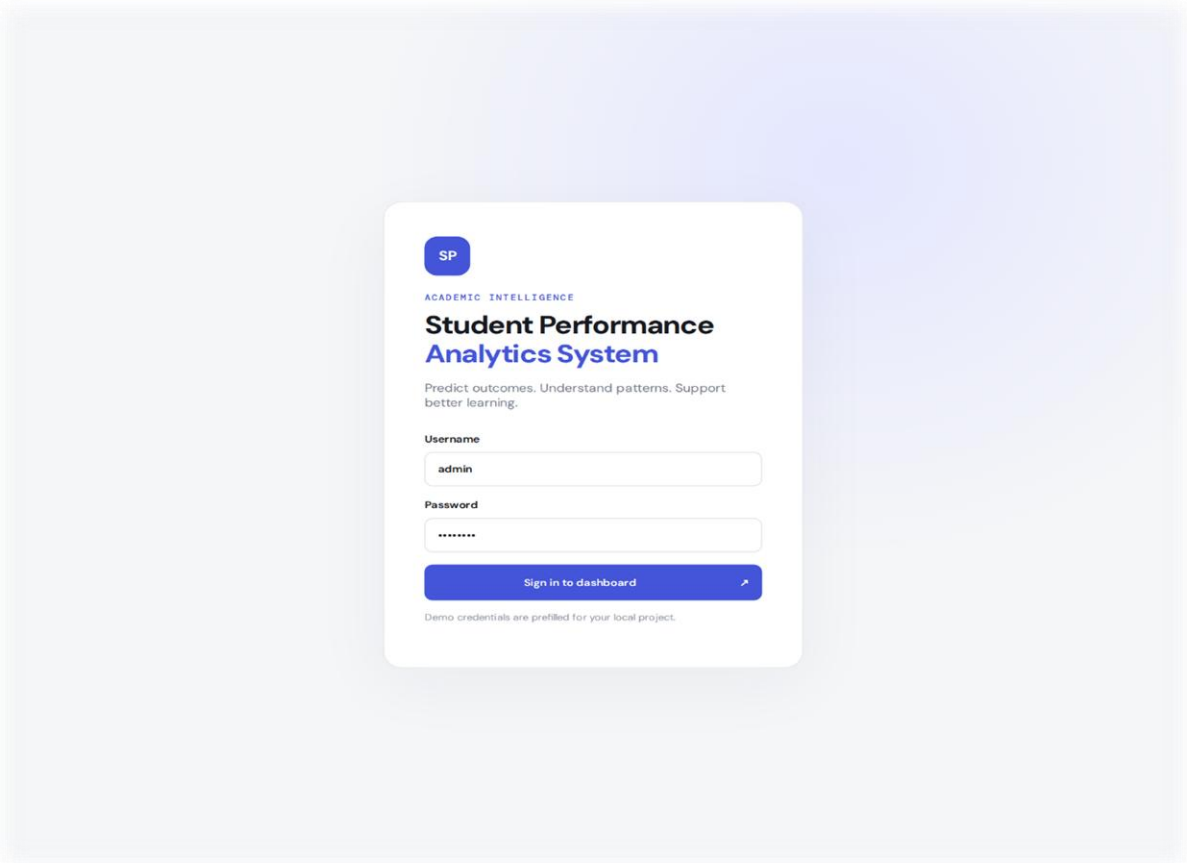


Figure 1: Login page with the demo credentials prefilled

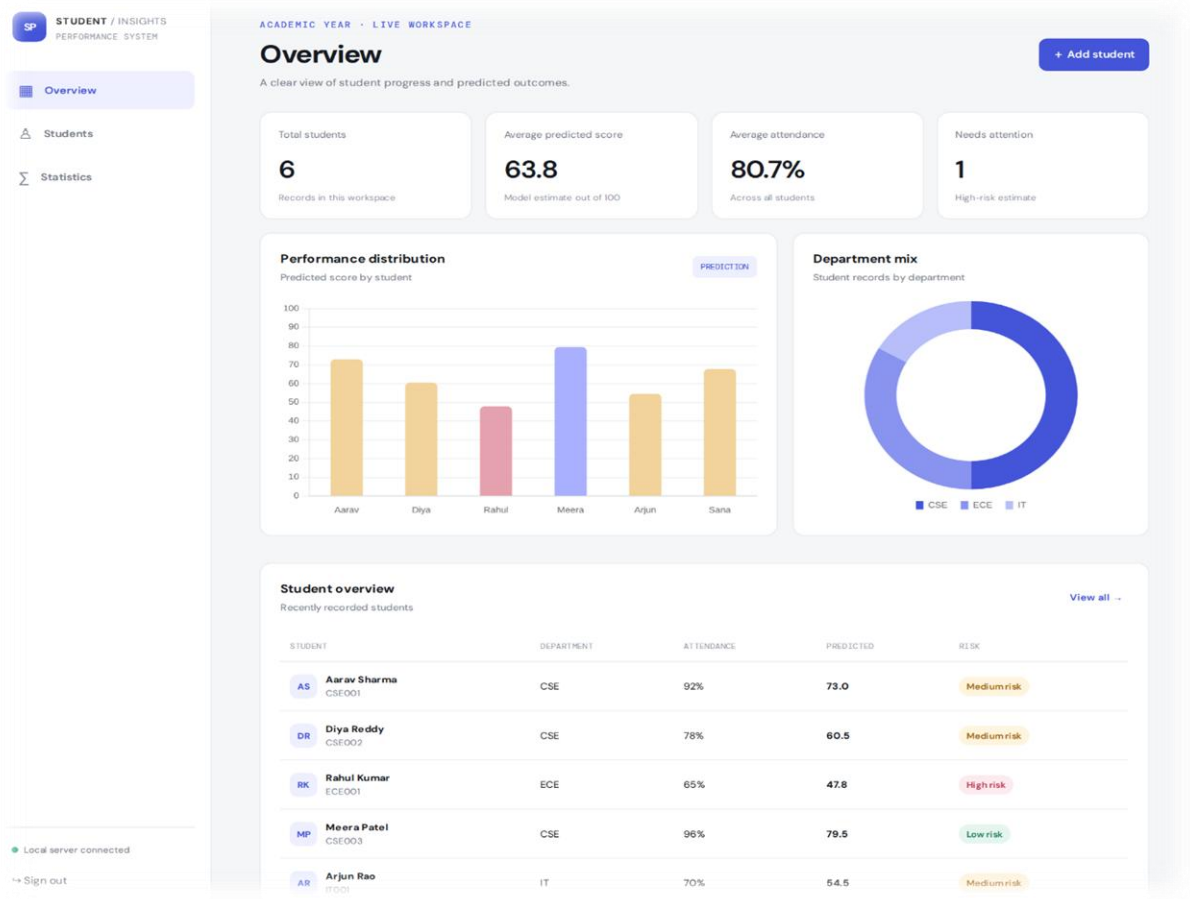


Figure 2: Overview dashboard for the 6 seeded students, with the predicted score chart and department mix

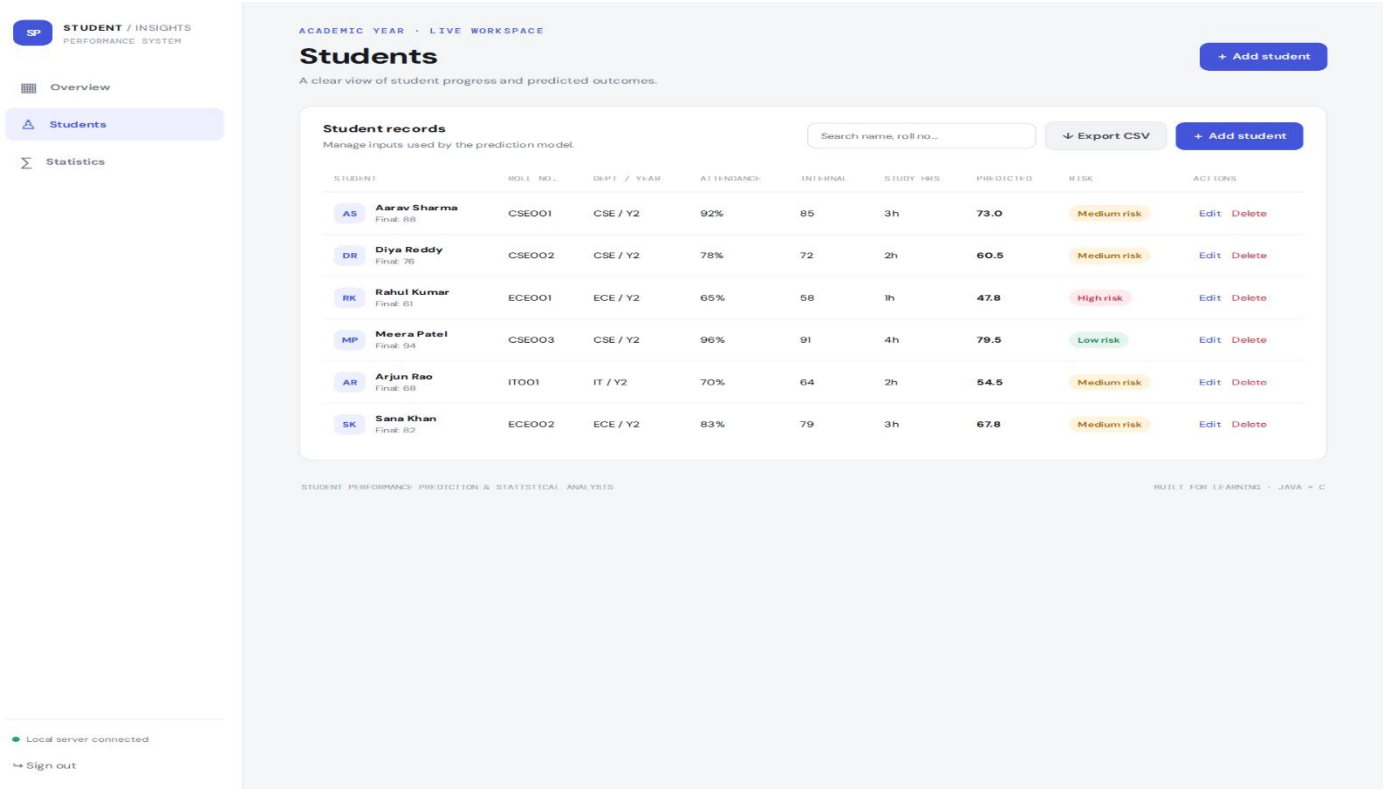


Figure 3: Students page with the searchable record table and risk levels

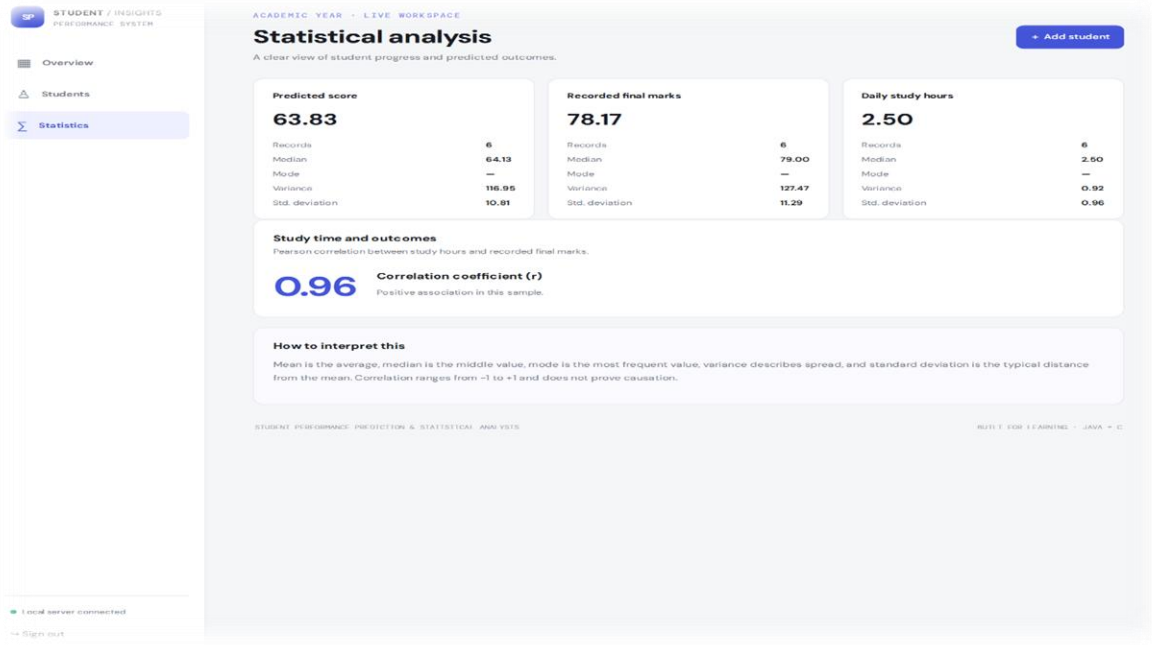


Figure 4: Statistics page with the mean, median, mode, variance, standard deviation and correlation (r = 0.96)

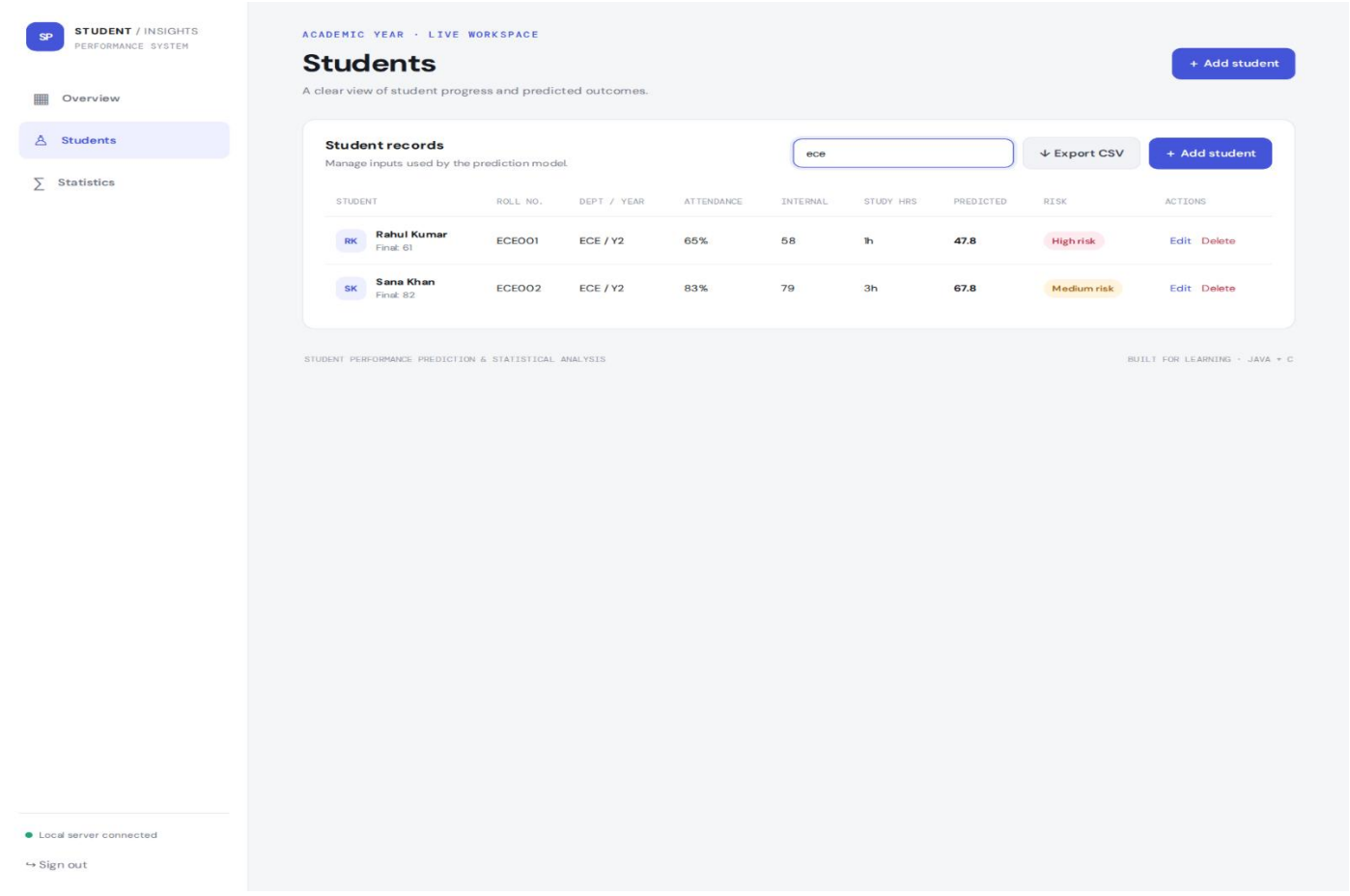


Figure 5: Search filtering the records by "ece"

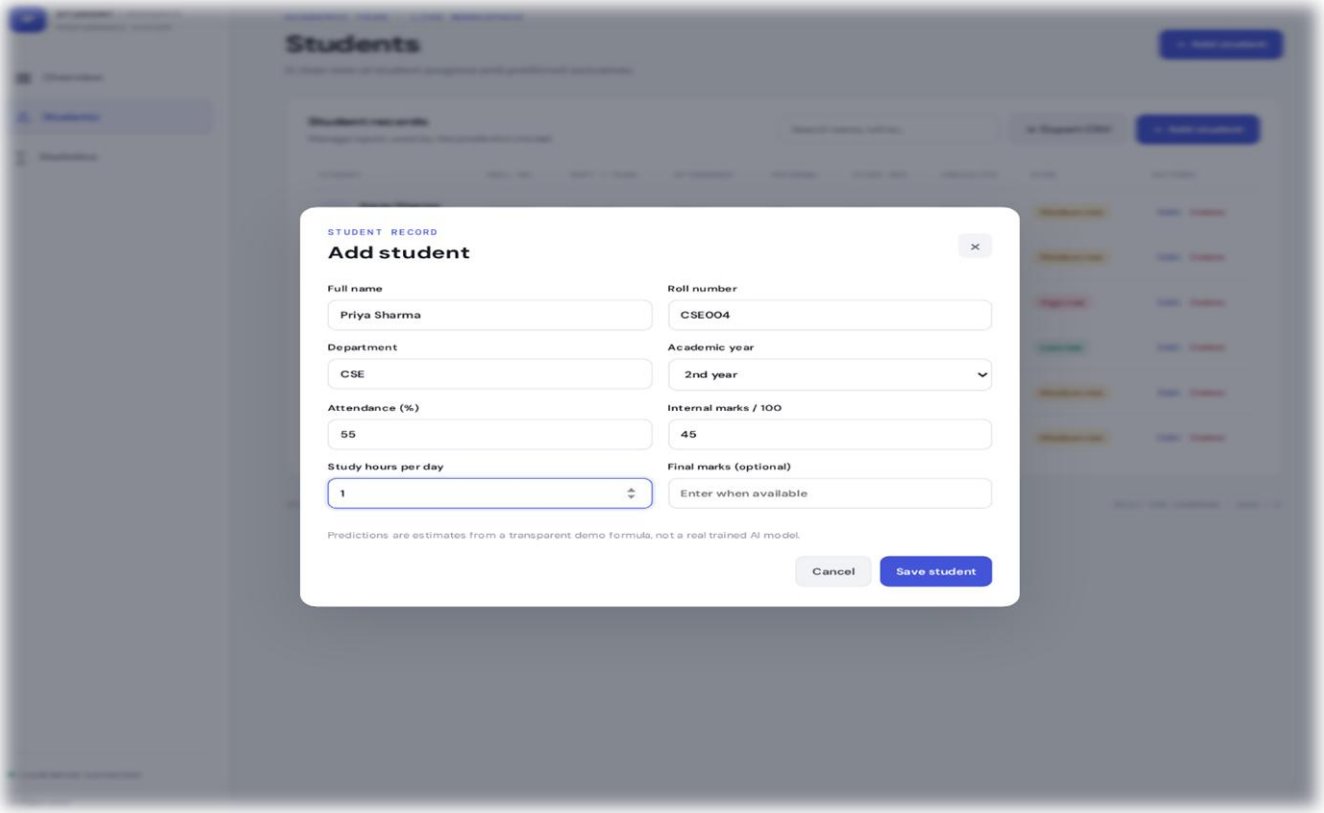


Figure 6: Add student form filled in with a new record

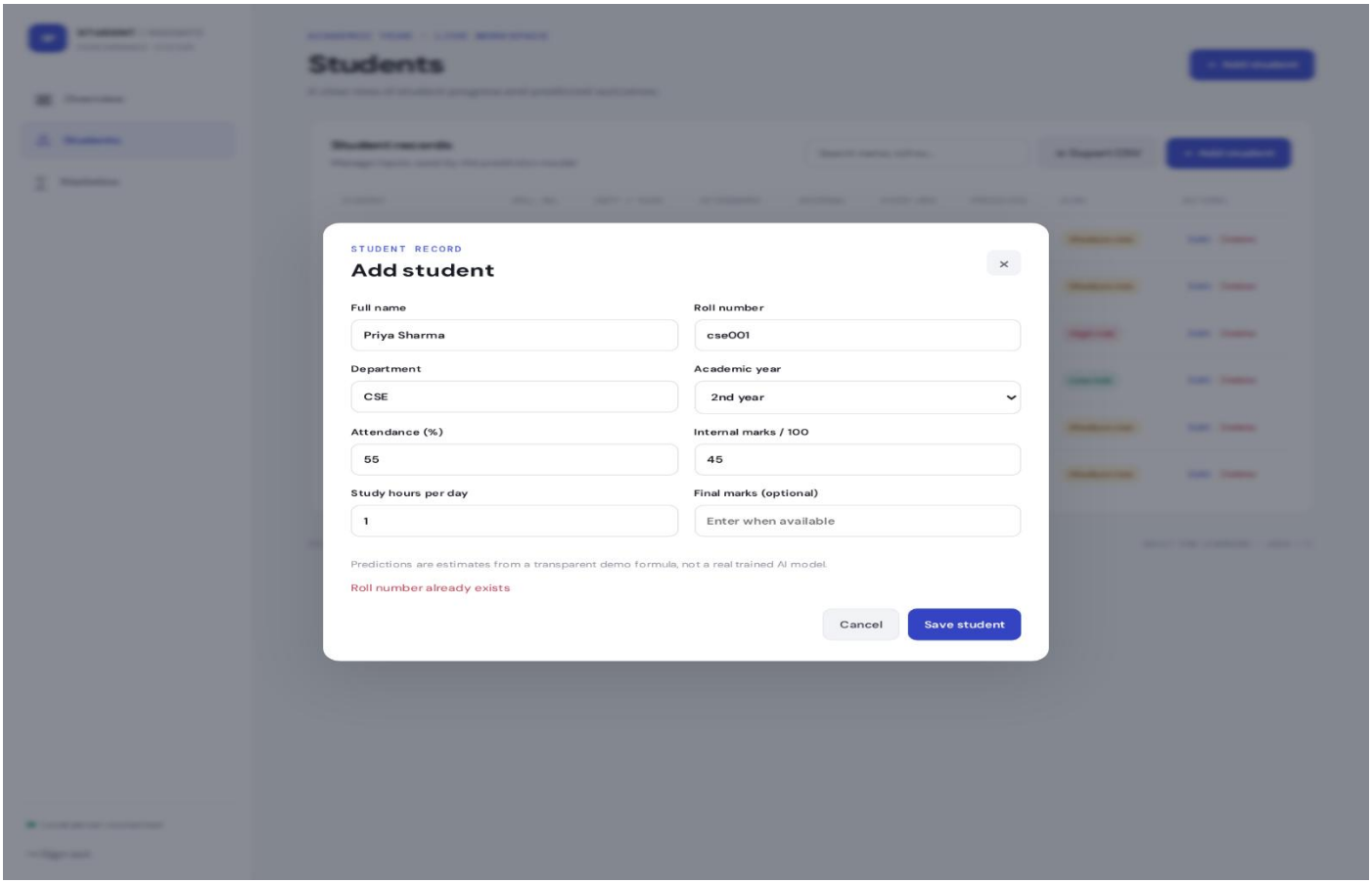


Figure 7: The server rejects a duplicate roll number with "Roll number already exists"

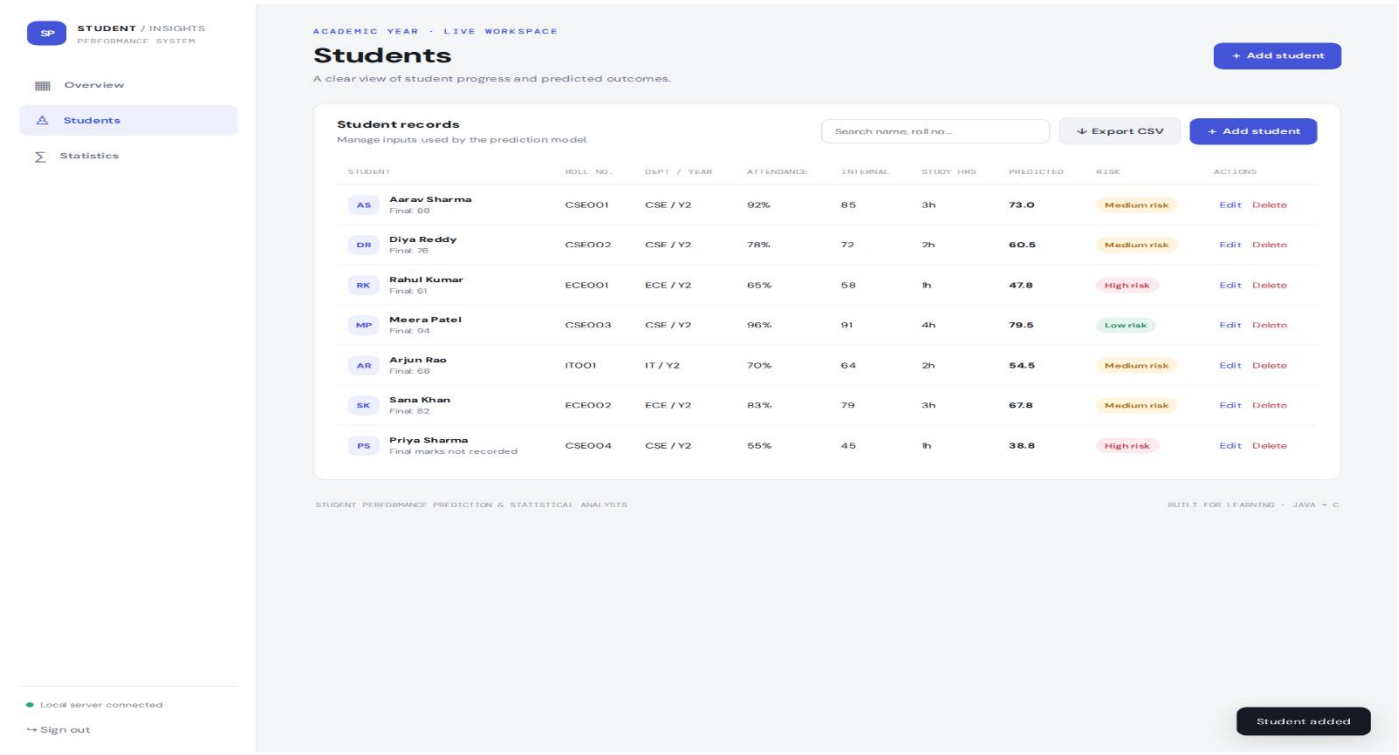


Figure 8: The new student appears in the table after saving

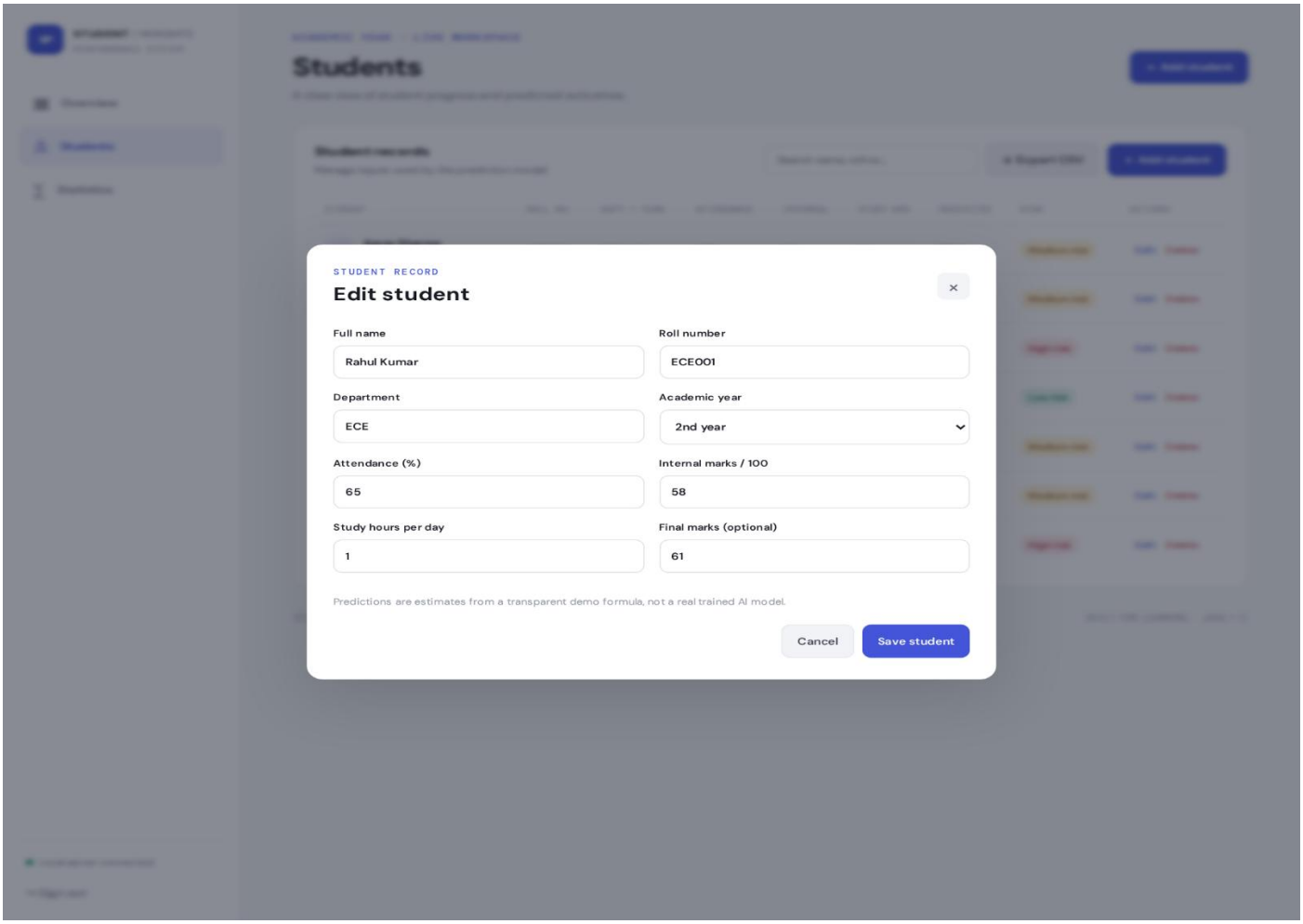


Figure 9: Edit student form for an existing record

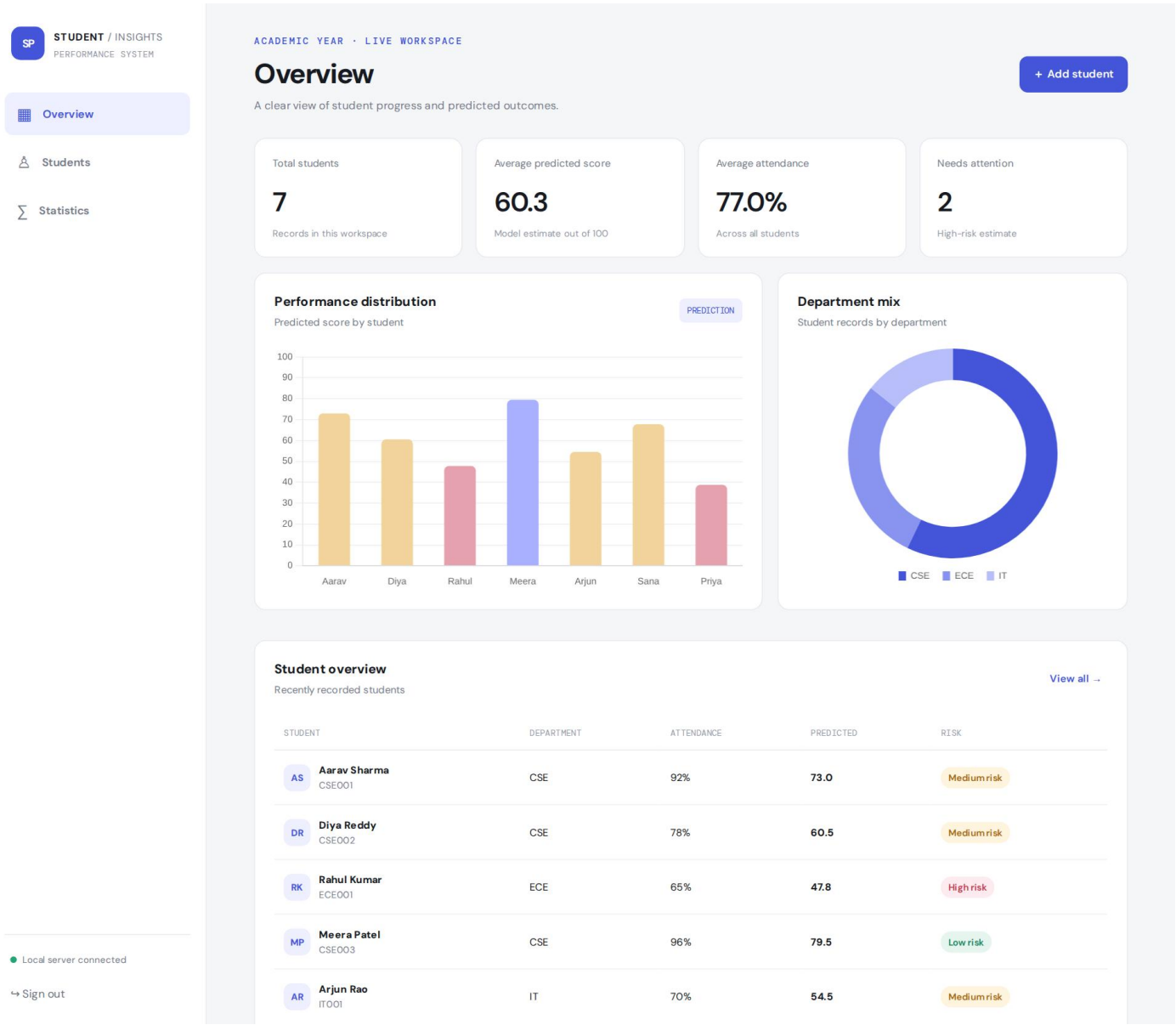


Figure 10: Dashboard after the new student is added: 7 students, average predicted score 60.3 and 2 students flagged as high risk

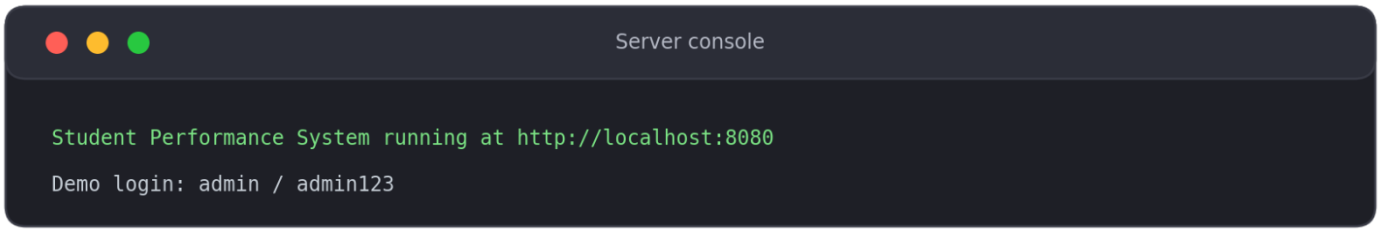


Figure 11: Server console after startup

C statistics module

```
$ gcc c-stats/statistics.c -o statistics -lm
$ ./statistics
STUDENT PERFORMANCE - C STATISTICS MODULE
---------------------------------------
Sample count          : 6
Mean                  : 78.17
Median                : 79.00
Mode                  : no unique mode
Population variance   : 127.47
Standard deviation    : 11.29
Study/final correlation: 0.964

Edit the arrays in main() to test other data.
```

Figure 12: C statistics module compiled with gcc and run

5. CONCLUSION

The project “Student Performance Prediction & Statistical Analysis System” shows how simple prediction and descriptive statistics can be combined in a full-stack application to support early identification of at-risk students. Student records are entered through a web interface, validated by a Java server and stored in a CSV file. The server then calculates a predicted score and risk level for each student, together with the mean, median, mode, variance, standard deviation and correlation. The results are shown on a dashboard with charts, and the same statistical calculations are implemented separately in C so that the two implementations can be compared.

The project gave practical experience with Java’s built-in HTTP server, REST APIs, session cookies, server-side validation, CSV file handling, Chart.js, descriptive statistics and the Pearson correlation. Automated testing confirmed the formula, the validation and the statistics, and it also found a defect: editing a student resets the record’s creation time. The project also showed the limits of the design. The prediction is a hand-weighted heuristic and not a trained model, and on the sample data it runs about 14 points below the actual final marks. Six students are too few for strong conclusions. The login is a hardcoded demo, traffic is plain HTTP, the in-memory student list is not thread-safe, and the C program works only on hardcoded sample arrays.

Future work includes fitting the weights with linear regression on real final marks and evaluating the model on separate test data, replacing the CSV file with a database, hashing passwords with session expiry and role-based access, HTTPS, thread-safe storage, allowing fractional study hours, tracking each student’s progress over time, and making the C program read students.csv.

6. REFERENCES

- D. C. Montgomery and G. C. Runger, *Applied Statistics and Probability for Engineers*, Wiley.
- B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, Prentice Hall.
- K. Pearson, "Notes on Regression and Inheritance in the Case of Two Parents," *Proceedings of the Royal Society of London*, vol. 58, pp. 240–242, 1895.
- P. Cortez and A. Silva, "Using Data Mining to Predict Secondary School Student Performance," *Proceedings of the 5th FUture BUsiness TEChnology Conference (FUBUTEC)*, 2008.
- RFC 9110 – HTTP Semantics, IETF.
- RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format, IETF.
- RFC 6265 – HTTP State Management Mechanism (Cookies), IETF.
- RFC 4180 – Common Format and MIME Type for Comma-Separated Values (CSV) Files, IETF.
- Oracle, Java SE documentation, com.sun.net.httpserver package, https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/package-summary.html
- Chart.js documentation, https://www.chartjs.org/docs/
- Mozilla, MDN Web Docs: Fetch API, https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Program Outcomes (POs)

PO1. Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2. Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3. Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4. Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5. Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6. The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7. Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8. Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9. Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10. Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11. Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

PROGRAM SPECIFIC OUTCOMES (PSOs):

PSO I: Apply appropriate data structures, analyze and design algorithms to solve complex problems using modern technologies in developing innovative solutions catering to societal needs.

PSO II: Apply Cryptographic Principles to analyze and develop cyber physical systems.

PSO III: Analyze the existing legal framework for cyber security and suggest strategies for protection of digital assets of an organization.

Note : Keep A tick (✓) Mark wherever there is a Mapping with Project

Title of the Project	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 1	PO 2	PO 3
Student Performance Prediction & Statistical Analysis System.	✓	✓	✓	✓	✓				✓		✓	✓		

SUSTAINABLE DEVELOPMENT GOALS (SDG):

1. No Poverty
2. No Hunger
3. Good Health
4. Quality Education
5. Gender Equality
6. Clean Water
7. Affordable and Clean Energy
8. Decent Work and Economic Growth
9. Industry, Innovation and infrastructure
10. Reduce inequalities
11. Sustainable Cities and communities
12. Responsible Consumption and Production
13. Climate Action
14. Life below water
15. Life on land
16. Peace, Justice and Strong Institutions
17. Partnerships for the Goals

Note : Keep A tick (✓) Mark wherever there is a Mapping with Project

Title of the Project	SDG 1	SDG 2	SDG 3	SDG 4	SDG 5	SDG 6	SDG 7	SDG 8	SDG 9	SDG 10	SDG 11	SDG 12	SDG 13	SDG 14	SDG 15	SDG 16	SDG 17
Student Performance Prediction & Statistical Analysis System.				✓					✓								

Project Summary



Geethanjali College of Engineering and Technology (Autonomous)

Accredited by NAAC with A+ Grade; B.Tech. CSE, EEE, ECE accredited by NBA

Sy. No: 33 & 34, Cheeryal (V), Keesara (M), Medchal District, Telangana – 501301

Team Number :

Roll Number : 24R11A6236 Name : Rithsika Sulakhe

Roll Number : 24R11A6258 Name : D. Spandana

Roll Number : 24R11A6262 Name : Gaddam Navya

Abstract :

ABSTRACT

The Student Performance Prediction & Statistical Analysis System is a web-based application that predicts student performance using internal marks, attendance, and study hours. It classifies students into Low, Medium, and High risk categories and calculates statistical measures such as mean, median, mode, variance, standard deviation, and correlation. Developed using Java 17, HTML, CSS, JavaScript, and Chart.js, the system supports student record management, dashboard visualization, and CSV export. It helps educators identify students who may need academic support.

Title Of the Project	Mapping - POs and PSOs	Mapping - SDGs
Student Performance Prediction & Statistical Analysis System.	PO1, PO2, PO3, PO4, PO5, PO9, PO11 and PSO I	SDG 4 and SDG 9

GITHUB LINK :

https://github.com/24r11a6236/Student-Performance-Prediction-Statistical-Analysis-System.git